

Class Activity: Smith-Waterman Algorithm for Local Sequence Alignment

Activity Title: Local Sequence Alignment using the Smith-Waterman Algorithm

Learning Objectives:

By the end of this activity, students should be able to:

- ✓ Understand the **Smith-Waterman algorithm** for local sequence alignment.
 - ✓ Implement the algorithm using **Biopython**.
 - ✓ Perform alignment of two DNA or protein sequences.
 - ✓ Solve exercises related to **local alignment scoring and traceback**.
-

Activity Overview

This class activity consists of **three parts**:

1. **Concept Explanation** – Understanding the Smith-Waterman algorithm.
 2. **Hands-on Coding Exercise** – Implementing and running alignments using Biopython.
 3. **Group Challenge & Exercises** – Solving alignment problems in teams and analyzing results.
-

Part 1: Concept Explanation (15 minutes)

1. Introduction to Local Sequence Alignment

- **Sequence alignment** compares two sequences to find similarities.
- **Local alignment** (Smith-Waterman) finds the **best matching subsequence** between two sequences.
- Used for **finding conserved regions** in DNA, RNA, or protein sequences.\

2. The Smith-Waterman Algorithm

The **Smith-Waterman** algorithm follows four main steps:

1. **Initialization** – Create a scoring matrix with zero values.
2. **Filling the Matrix** – Compute scores based on matches, mismatches, gaps.
3. **Traceback** – Identify the highest-scoring **local** alignment.
4. **Result Extraction** – Report the aligned subsequences.

Scoring System (Example)

- **Match:** +2
- **Mismatch:** -1
- **Gap penalty:** -2

Example Sequences:

Let's align "TGTTACGG" and "GGTTGACTA" using Smith-Waterman.

Optimal Local Alignment:

Part 2: Hands-on Coding Exercise (20 minutes)

1. Implementing Smith-Waterman in Biopython

```
from Bio import pairwise2
from Bio.pairwise2 import format_alignment

# Define sequences
seq1 = "TGTTACGG"
seq2 = "GGTTGACTA"

# Perform local alignment
alignments = pairwise2.align.localxx(seq1, seq2)

# Print results
for alignment in alignments:
    print(format_alignment(*alignment))
```

 **Expected Output:**

2. Custom Scoring System

Modify the alignment to use **match = 2, mismatch = -1, gap = -2**.

```
alignments = pairwise2.align.localms(seq1, seq2, 2, -1, -2, -2)
```

for alignment in alignments:

```
    print(format_alignment(*alignment))
```

3. User Input Sequences

Ask students to modify the script to:

1. Take **two sequences as user input**.
2. Perform **local alignment using localxx (default scoring)**.
3. Print **the best alignment only**.

Hint: Use `input()` to collect sequences.

Part 3: Group Challenge & Exercises (25 minutes)



Group Challenge (10 minutes)

1. Form small groups (2-3 students).
2. Each group is given **two sequences** to align.
3. They must **calculate the alignment matrix manually** for a small sequence pair.
4. **Verify results** by running their calculations in Biopython.

Example Sequences for Groups

- **Group 1:** "ATGC" vs. "AGC"
- **Group 2:** "GACT" vs. "GAT"
- **Group 3:** "TGCAG" vs. "TGAC"
- **Group 4:** "ACGTAC" vs. "ACTGAC"

Discussion: How do **gap penalties** affect the alignment?



Individual Exercises (15 minutes)

Exercise 1: Identify Best Local Alignment

For "AGCTA" and "GCT", compute the **best local alignment score** manually using **match = 2**, **mismatch = -1**, **gap = -2**.

Exercise 2: Custom Alignment Function

Write a Python function that:

- Takes **two sequences** as input.
- Uses `pairwise2.align.localms()` with **custom scoring**.
- Returns the **best local alignment only**.

Exercise 3: Effect of Scoring System

Run alignments with different **gap penalties** (-1, -2, -3, -5) and analyze how the results change.

Expected Learning Outcomes

- ✓ Understand the **Smith-Waterman algorithm** conceptually and practically.
 - ✓ Implement local sequence alignment using **Biopython**.
 - ✓ Interpret **alignment scores and traceback results**.
 - ✓ Analyze the effects of **different scoring systems**.
-

Discussion Questions

1. Why does increasing the **gap penalty** lead to shorter alignments?
 2. What happens when sequences **do not align well**?
 3. How does this algorithm differ from **global alignment (Needleman-Wunsch)**?
-

Additional Resources

- **Biopython Documentation:** <https://biopython.org/wiki/Documentation>
- **NCBI Resources:** <https://www.ncbi.nlm.nih.gov/>